

This project is a **full-stack Placement Tracker web application**. It allows a user to add, view, edit, and delete job/internship applications, with the data permanently stored in PostgreSQL.

The project is built with a **separate frontend and backend**. The frontend is a Vue.js single-page application. It is responsible for the user interface, forms, navigation, displaying applications, and sending HTTP requests. It does not directly access the database.

The backend is an **ASP.NET Core Web API written in C#**. It exposes REST API endpoints such as creating, reading, updating, and deleting applications. The backend receives JSON from Vue, validates the data, performs the required operation, and returns JSON responses with appropriate HTTP status codes.

For database operations, the backend uses **Entity Framework Core** as the ORM. EF Core converts the C# application/entity operations into database operations. The actual database is **PostgreSQL**, accessed through the **Npgsql Entity Framework Core provider**.

The frontend communicates with the backend using the browser's **Fetch API** and JSON. The Vue application has an API service layer so the components don't directly contain all the HTTP request logic.

The application also has backend validation and frontend validation. The backend is ultimately responsible for protecting the data and enforcing the application's rules.

The project includes automated testing using **xUnit**, **ASP.NET Core integration testing**, **Entity Framework Core InMemory** for isolated test scenarios, and **Moq** for mocking where required.

Tech stack — Summarised

Frontend

- **Vue 3:** JavaScript framework for building the user interface.
- **Vue Router:** Handles navigation between frontend pages.
- **JavaScript:** Programming language used for the frontend logic.
- **Vite:** Development server and build tool for Vue.
- **Fetch API:** Sends HTTP requests from Vue to the backend.
- **HTML:** Defines the structure of the web pages.
- **CSS:** Controls the appearance and styling.

Backend

- **C#:** Programming language used for the backend.
- **ASP.NET Core 8 Web API:** Framework for building the backend REST API.
- **REST/JSON:** Communication format between Vue and the backend.
- **ControllerBase:** Base class used to create API controllers and handle HTTP requests.

Database

- **PostgreSQL:** Relational database that permanently stores application data.

ORM

- **Entity Framework Core 8:** ORM that lets C# code interact with PostgreSQL without writing most SQL manually.

Database Provider

- **Npgsql:** Connects Entity Framework Core/ASP.NET Core to PostgreSQL.

Testing

- **xUnit:** Framework for writing automated C# tests.
- **EF Core InMemory:** Provides an isolated in-memory database for tests.
- **Moq:** Creates mock objects for testing specific components.

Package Management

- **npm:** Installs and manages Vue/JavaScript packages.
- **NuGet:** Installs and manages C#/.NET packages.

UNDERSTANDING THE CODE (KNOWLEDGE BYTES)

bin stands for **Binary**.

In your .NET project, the bin folder contains the **compiled/build output** of your application.

In simple terms:

Source code (.cs) → Build → bin → runnable application files

It typically contains:

- Compiled .dll files
- Executable files
- Required .NET libraries/dependencies
- Different builds such as Debug or Release

You **normally don't edit anything inside bin manually**. Visual Studio/.NET automatically creates and updates it when you build or run the project.

ApplicationsController.cs

Byte 1: What ApplicationsController is responsible for

Builds on: None — starting point

In plain terms:

This controller is the **API entry point for managing placement applications**. It receives HTTP requests from the frontend and performs CRUD operations — **Create, Read, Update, Delete** — using the database.

The code:

```
[ApiController]
```

```
[Route("api/applications")]
```

```
public class ApplicationsController : ControllerBase
```

What's happening:

[ApiController] tells ASP.NET Core that this is a Web API controller.

[Route("api/applications")] means all endpoints start with /api/applications.

Why it matters:

This is the bridge between your **Vue frontend** and your **database**. The frontend doesn't directly access PostgreSQL; it communicates through this controller.

Byte 2: Connecting the Controller to the Database

Builds on: Byte 1

In plain terms:

The controller needs a way to communicate with the database. ApplicationDbContext is the EF Core object that provides that connection.

The code:

```
private readonly ApplicationDbContext _db;

public ApplicationController(ApplicationDbContext db)
{
    _db = db;
}
```

What's happening:

ASP.NET Core automatically provides an ApplicationDbContext when creating the controller. This is called **dependency injection**. The controller stores it in `_db` so every method can use the database.

Why it matters:

Without `_db`, the controller couldn't retrieve, create, update, or delete applications.

Byte 3: Cleaning Optional Text

Builds on: Byte 2

In plain terms:

Some fields, such as JobUrl and Notes, are optional. This helper converts empty input into null and removes unnecessary spaces.

The code:

```
private static string? NormalizeOptional(string? value) =>
    string.IsNullOrEmpty(value) ? null : value.Trim();
```

What's happening:

For example:

```
" https://company.com " → "https://company.com"
"   " → null
```

Why it matters:

It keeps your database cleaner and avoids storing meaningless whitespace as actual data.

Byte 4: Getting All Applications

Builds on: Byte 2

In plain terms:

The GetAll() method handles a **GET request** and returns every placement application from the database.

The code:

[HttpGet]

```
public async Task<ActionResult<IEnumerable<ApplicationResponseDto>>> GetAll()
{
    var apps = await _db.Applications
        .OrderByDescending(a => a.Id)
        .ToListAsync();

    return Ok(apps.Select(ApplicationResponseDto.FromEntity).ToList());
}
```

What's happening:

It retrieves applications, puts the newest IDs first, converts the database entities into response DTOs, and returns HTTP **200 OK** with JSON.

Why it matters:

This is what the frontend uses when it needs to **display the list of applications**.

Byte 5: Getting One Application

Builds on: Byte 4

In plain terms:

GetOne() retrieves a specific application using its ID.

The code:

[HttpGet("{id:int}")]

```
public async Task<ActionResult<ApplicationResponseDto>> GetOne(int id)
{
    var app = await _db.Applications.FindAsync(id);

    if (app is null)
        return NotFound(new { message = $"Application {id} not found." });
}
```

```
    return Ok(ApplicationResponseDto.FromEntity(app));  
}
```

What's happening:

A request such as:

GET /api/applications/5

looks for application 5. If it exists, it returns **200 OK**. If it doesn't, it returns **404 Not Found**.

Why it matters:

The frontend needs this when viewing or editing a particular application.

Byte 6: Creating a New Application

Builds on: Bytes 2–5

In plain terms:

Create() handles a **POST request** and adds a new placement application to the database.

The code:

```
[HttpPost]  
public async Task<ActionResult<ApplicationResponseDto>> Create(  
    [FromBody] CreateApplicationDto dto)  
{  
    if (!ModelState.IsValid)  
        return ValidationProblem(ModelState);  
  
    var app = new Models.Application  
    {  
        Company = dto.Company.Trim(),  
        Role = dto.Role.Trim(),  
        Status = dto.Status,  
        AppliedOn = dto.AppliedOn,  
        JobUrl = NormalizeOptional(dto.JobUrl),  
        Notes = NormalizeOptional(dto.Notes),  
        CreatedAt = DateTime.UtcNow  
    }  
}
```

```
};
```

What's happening:

The JSON request is converted into `CreateApplicationDto`. Validation is checked first. Then a database `Application` object is created from the DTO.

Why it matters:

This is the main operation behind the frontend's "**Add Application**" functionality.

Byte 7: Saving the New Application

Builds on: Byte 6

In plain terms:

Creating the C# object isn't enough. It must be added to EF Core and saved to the actual database.

The code:

```
_db.Applications.Add(app);  
await _db.SaveChangesAsync();  
  
return CreatedAtAction(  
    nameof(GetOne),  
    new { id = app.Id },  
    ApplicationResponseDto.FromEntity(app));
```

What's happening:

`Add()` tells EF Core that a new record should be inserted. `SaveChangesAsync()` actually sends the change to PostgreSQL.

`CreatedAtAction()` returns **201 Created** and provides the newly created application.

Why it matters:

Without `SaveChangesAsync()`, the new application would exist only temporarily in memory and wouldn't be stored in the database.

Byte 8: Updating an Existing Application

Builds on: Bytes 5–7

In plain terms:

`Update()` handles a **PUT request**. It first finds the existing application, then replaces its editable values with the new ones.

The code:

```

var existing = await _db.Applications.FindAsync(id);

if (existing is null)
    return NotFound(new { message = $"Application {id} not found." });

existing.Company = dto.Company.Trim();
existing.Role = dto.Role.Trim();
existing.Status = dto.Status;
existing.AppliedOn = dto.AppliedOn;
existing.JobUrl = NormalizeOptional(dto.JobUrl);
existing.Notes = NormalizeOptional(dto.Notes);

await _db.SaveChangesAsync();

return Ok(ApplicationResponseDto.FromEntity(existing));

```

What's happening:

The existing database entity is found and its properties are changed. EF Core tracks those changes and saves them when `SaveChangesAsync()` runs.

Why it matters:

This powers the frontend's **Edit Application** functionality.

Byte 9: Deleting an Application

Builds on: Byte 5

In plain terms:

`Delete()` removes an application from the database using its ID.

The code:

```

[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id)
{
    var app = await _db.Applications.FindAsync(id);

    if (app is null)

```

```
        return NotFound(new { message = $"Application {id} not found." });

        _db.Applications.Remove(app);
        await _db.SaveChangesAsync();

        return NoContent();
    }
}
```

What's happening:

The application is found first. If it exists, EF Core marks it for deletion and `SaveChangesAsync()` permanently applies the deletion.

The successful response is **204 No Content**.

Why it matters:

This provides the backend functionality for the frontend's **Delete Application** operation.

Byte 10: Why the Controller Uses DTOs

Builds on: Bytes 4–8

In plain terms:

A **DTO (Data Transfer Object)** is a small class used specifically for data moving between the frontend and API. Your controller uses DTOs instead of directly exposing the database model.

The code:

CreateApplicationDto dto

UpdateApplicationDto dto

ApplicationResponseDto

What's happening:

There are separate DTOs for incoming create/update data and outgoing response data.

Why it matters:

This separates your **API contract** from your database structure. You can change the database model later without necessarily changing exactly what the frontend receives.

Byte 11: Validation and Error Responses

Builds on: Bytes 5–8

In plain terms:

The controller doesn't blindly accept every request. It checks validation rules and handles missing records with appropriate HTTP status codes.

The code:

```
if (!ModelState.IsValid)
{
    return ValidationProblem(ModelState);
}

and:

if (existing is null)
{
    return NotFound(new { message = $"Application {id} not found." });
}
```

What's happening:

Invalid input produces a validation error, while requesting an application that doesn't exist produces **404 Not Found**.

Why it matters:

This prevents bad data from reaching the database and gives the frontend useful information when something goes wrong.

ApplicationDbContext.cs

Byte 1: What ApplicationDbContext is responsible for

Builds on: None — starting point

In plain terms:

ApplicationDbContext is the **EF Core bridge between your C# application and PostgreSQL**. It lets the controllers work with database records using C# objects instead of writing SQL manually.

The code:

```
public class ApplicationDbContext : DbContext  
{  
}
```

What's happening:

DbContext is an Entity Framework Core class that manages database communication and tracks changes to data.

Why it matters:

Without this class, your controller wouldn't have the `_db.Applications` interface used to read, insert, update, and delete applications.

Byte 2: Configuring the Database Connection

Builds on: Byte 1

In plain terms:

The constructor receives the database configuration from ASP.NET Core and passes it to EF Core.

The code:

```
public ApplicationDbContext(  
    DbContextOptions<ApplicationDbContext> options)  
    : base(options)  
{  
}
```

What's happening:

DbContextOptions contains configuration such as **which database provider to use and how to connect to it**. `base(options)` passes that configuration to the parent DbContext.

Why it matters:

This is what allows the same ApplicationDbContext to work with your configured PostgreSQL database.

Byte 3: Representing the Applications Table

Builds on: Byte 1

In plain terms:

DbSet represents a collection of database records. Here, Applications represents the **Application table** in the database.

The code:

```
public DbSet<Models.Application> Applications
    => Set<Models.Application>();
```

What's happening:

EF Core uses the Application model to understand what an application record looks like. The controller can then do things such as:

```
_db.Applications.Add(app);
```

or

```
_db.Applications.FindAsync(id);
```

Why it matters:

This is the main doorway through which your controller accesses application records.

Byte 4: Customizing the Database Model

Builds on: Byte 3

In plain terms:

OnModelCreating() tells EF Core **how the C# Application model should be represented in the database**.

The code:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Models.Application>(entity =>
    {
        // configuration
    });
}
```

What's happening:

ModelBuilder lets you define database rules such as primary keys, required fields, maximum lengths, and column types.

Why it matters:

It gives you more precise database configuration than relying only on the C# model's default conventions.

Byte 5: Defining the Primary Key

Builds on: Byte 4

In plain terms:

Every application record needs a unique identifier. This configuration tells EF Core that Id is the **primary key**.

The code:

```
entity.HasKey(a => a.Id);
```

What's happening:

The Id property uniquely identifies each row in the Applications table.

For example:

Id	Company	Role
1	Google	ML Engineer
2	Microsoft	Software Engineer
3	Amazon	Data Analyst

Why it matters:

Operations such as FindAsync(id) depend on having a unique key to identify the correct record.

Byte 6: Required Fields and Maximum Length

Builds on: Byte 5

In plain terms:

The database is given rules about which fields are mandatory and how much text they can contain.

The code:

```
entity.Property(a => a.Company)
    .IsRequired()
    .HasMaxLength(100);
```

```
entity.Property(a => a.Role)
    .IsRequired()
    .HasMaxLength(100);
```

```
entity.Property(a => a.Status)
    .IsRequired()
    .HasMaxLength(20);
```

What's happening:

IsRequired() means the field cannot be empty/null at the database level. HasMaxLength() limits how much text can be stored.

Why it matters:

These constraints help prevent invalid or excessively large data from being stored in PostgreSQL.

Byte 7: Limiting the Notes Field

Builds on: Byte 6

In plain terms:

Notes is optional, but it still has a maximum size.

The code:

```
entity.Property(a => a.Notes)
    .HasMaxLength(1000);
```

What's happening:

Unlike Company and Role, there is no IsRequired(), so Notes can be null. However, if provided, it can contain at most 1000 characters.

Why it matters:

This allows users to leave notes empty while preventing unnecessarily large text from being stored.

Byte 8: Choosing the Database Date/Time Type

Builds on: Byte 7

In plain terms:

The application explicitly tells PostgreSQL how to store its date/time values: **without timezone information**.

The code:

```
entity.Property(a => a.AppliedOn)
    .HasColumnType("timestamp without time zone");
```

```
entity.Property(a => a.CreatedAt)
    .HasColumnType("timestamp without time zone");
```

What's happening:

Both `AppliedOn` and `CreatedAt` are stored as PostgreSQL timestamp without time zone values.

The comment explains that this avoids problems when C# `DateTime` values have an `Unspecified` kind, which can conflict with PostgreSQL's timestamp with time zone handling through `Npgsql`.

Why it matters:

This prevents date/time type mismatches between **C# `DateTime`** → **EF Core** → **Npgsql** → **PostgreSQL**.

ApplicationDtos.cs

Byte 1: What this DTO file is responsible for

Builds on: None — starting point

In plain terms:

This file defines the **data shapes used by the API** when creating, updating, and returning placement applications. These classes are called **DTOs (Data Transfer Objects)** because they carry data between the frontend and backend.

The code:

```
namespace PlacementTracker.Api.DTOs;
```

```
public class CreateApplicationDto
```

```
{  
    // data coming into the API  
}
```

```
public class UpdateApplicationDto
```

```
{  
    // data coming into the API  
}
```

```
public class ApplicationResponseDto
```

```
{  
    // data going back to the frontend  
}
```

What's happening:

Instead of directly sending the database Application model everywhere, the API uses separate objects for **input and output**.

Why it matters:

It keeps the API's data contract separate from the database structure and gives you a place to apply input validation.

Byte 2: Centralizing Allowed Application Statuses

Builds on: Byte 1

In plain terms:

The application can only have certain statuses. ApplicationStatus stores those allowed values in one shared location.

The code:

```
public static class ApplicationStatus
{
    public static readonly string[] Allowed =
        new[] { "Wishlist", "Applied", "Interview", "Offer", "Rejected" };
}
```

What's happening:

The Allowed array acts as the list of valid statuses. Other code can check whether a supplied status exists in this list.

Why it matters:

It prevents different parts of the application from using different sets of valid statuses.

Byte 3: The Create DTO

Builds on: Bytes 1–2

In plain terms:

CreateApplicationDto describes the information the frontend must send when creating a new application.

The code:

```
public class CreateApplicationDto : IValidatableObject
{
    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = "Applied";
    public DateTime? AppliedOn { get; set; }
    public string? JobUrl { get; set; }
    public string? Notes { get; set; }
}
```

What's happening:

It contains the fields needed to create an application. ? means a value can be null, so AppliedOn, JobUrl, and Notes are optional at the type level.

Why it matters:

The controller can receive a predictable structure instead of accepting arbitrary JSON.

Byte 4: Basic Validation Attributes

Builds on: Byte 3

In plain terms:

The DTO uses **Data Annotations** to define simple validation rules directly on its properties.

The code:

```
[Required(ErrorMessage = "Company is required.")]
[StringLength(100, MinimumLength = 2,
    ErrorMessage = "Company must be 2-100 characters.")]
public string Company { get; set; } = string.Empty;
```

What's happening:

[Required] prevents a missing value, while [StringLength] requires the company name to be between **2 and 100 characters**.

Similar rules are applied to Role, Notes, and JobUrl.

Why it matters:

ASP.NET Core can automatically detect many invalid inputs before the controller performs the database operation.

Byte 5: URL Validation

Builds on: Byte 4

In plain terms:

The JobUrl field has a specific validation rule because it is expected to contain a web address.

The code:

```
[Url(ErrorMessage =
    "Please enter a valid URL (e.g. https://example.com/job).")]
public string? JobUrl { get; set; }
```

What's happening:

ASP.NET Core's validation system checks whether the supplied value matches the expected URL format.

Why it matters:

It prevents obviously malformed URLs from being accepted as job links.

Byte 6: Custom Validation with IValidatableObject

Builds on: Bytes 2–5

In plain terms:

Some rules depend on **multiple fields**, so attributes alone aren't enough. IValidatableObject allows the DTO to define its own custom validation logic.

The code:

```
public class CreateApplicationDto : IValidatableObject
{
    public IEnumerable<ValidationResult> Validate(
        ValidationContext validationContext)
    {
        // custom rules
    }
}
```

What's happening:

ASP.NET Core calls Validate() as part of model validation. The method can return one or more ValidationResult objects describing problems.

Why it matters:

This lets you enforce business rules such as **"AppliedOn is required unless the status is Wishlist."**

Byte 7: Validating the Status

Builds on: Byte 2 and Byte 6

In plain terms:

The custom validation makes sure the submitted status is one of the application's approved statuses.

The code:

```
if (!ApplicationStatus.Allowed.Contains(Status))
{
    yield return new ValidationResult(
        "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected.",
        new[] { nameof(Status) });
}
```

What's happening:

Contains() checks whether Status exists in the shared Allowed array. If it doesn't, a validation error is returned and associated with the Status field.

Why it matters:

Without this check, someone could send values such as "RandomStatus" that don't belong to the application's workflow.

Byte 8: Validating Company and Role

Builds on: Byte 6

In plain terms:

The custom validation also checks the **actual trimmed content** of Company and Role, not just whether they technically contain a string.

The code:

```
if (string.IsNullOrEmpty(Company) ||
    Company.Trim().Length < 2)
{
    yield return new ValidationResult(
        "Company must be 2-100 characters.",
        new[] { nameof(Company) });
}
```

What's happening:

A value such as:

" "

is considered invalid. Likewise, a company name containing only one meaningful character fails the minimum-length requirement.

The same idea is applied to Role.

Why it matters:

It catches inputs that might pass a basic type check but are not meaningful data.

Byte 9: The Conditional Applied-Date Rule

Builds on: Bytes 6–8

In plain terms:

The application has a special business rule: an application in the **Wishlist** stage doesn't need an application date, but other statuses do.

The code:

```
if (Status != "Wishlist" && AppliedOn is null)
{
    yield return new ValidationResult(
        "Applied date is required unless status is Wishlist.",
        new[] { nameof(AppliedOn) });
}
```

What's happening:

Two conditions must be true:

- Status is **not** Wishlist
- AppliedOn is missing

If both happen, validation fails.

Why it matters:

This is a good example of why custom validation exists: the validity of one field depends on another field.

Byte 10: Why UpdateApplicationDto Exists Separately

Builds on: Byte 3

In plain terms:

UpdateApplicationDto represents the data sent when **editing an existing application**.

The code:

```
public class UpdateApplicationDto : IValidatableObject
{
    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = "Applied";
    public DateTime? AppliedOn { get; set; }
    public string? JobUrl { get; set; }
    public string? Notes { get; set; }
}
```

What's happening:

It contains essentially the same fields and validation rules as `CreateApplicationDto`, but it represents a different API operation: **updating rather than creating**.

Why it matters:

Keeping create and update requests separate gives you flexibility to change their rules independently later.

Byte 11: The Response DTO

Builds on: Byte 1

In plain terms:

`ApplicationResponseDto` defines the data the API sends **back to the frontend** after reading or modifying an application.

The code:

```
public class ApplicationResponseDto
{
    public int Id { get; set; }
    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = string.Empty;
    public DateTime? AppliedOn { get; set; }
    public string? JobUrl { get; set; }
    public string? Notes { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

What's happening:

This DTO contains the information the frontend needs, including the generated `Id` and `CreatedAt`.

Why it matters:

The API controls exactly what data is exposed to the frontend instead of automatically returning the entire database entity.

Byte 12: Converting the Database Entity to a Response DTO

Builds on: Byte 11

In plain terms:

FromEntity() converts a database Application object into an ApplicationResponseDto.

The code:

```
public static ApplicationResponseDto FromEntity(
    Models.Application app) => new()
{
    Id = app.Id,
    Company = app.Company,
    Role = app.Role,
    Status = app.Status,
    AppliedOn = app.AppliedOn,
    JobUrl = app.JobUrl,
    Notes = app.Notes,
    CreatedAt = app.CreatedAt
};
```

What's happening:

It copies the required values from the database model into the response DTO.

The controller can then do:

```
ApplicationResponseDto.FromEntity(app)
```

Why it matters:

It provides one consistent way to transform database entities into API responses instead of repeating the mapping code throughout the controller.

Application.cs

Byte 1: What the Application model represents

Builds on: None — starting point

In plain terms:

Application is the main **data model** for one job or internship application. Each object represents one application record that will eventually be stored as a row in PostgreSQL.

The code:

```
public class Application : IValidatableObject
{
    public int Id { get; set; }

    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = "Applied";
}
```

What's happening:

The class defines the basic information an application contains: an ID, company, role, and status. IValidatableObject also allows the model to define custom validation rules.

Why it matters:

This model is the central representation of an application across the backend and database.

Byte 2: The Application ID

Builds on: Byte 1

In plain terms:

Id uniquely identifies each application. It acts as the **primary key** for the database record.

The code:

```
public int Id { get; set; }
```

What's happening:

Each application gets a numeric identifier such as 1, 2, or 3. The ApplicationDbContext later configures this property as the database primary key.

Why it matters:

The API uses the ID to find, update, or delete a specific application, such as:

GET /api/applications/5

Byte 3: Company and Role Fields

Builds on: Byte 1

In plain terms:

These fields identify **where the person applied** and **what position they applied for**. Both have basic validation rules.

The code:

```
[Required(ErrorMessage = "Company is required.")]
```

```
[StringLength(100, MinimumLength = 2)]
```

```
public string Company { get; set; } = string.Empty;
```

```
[Required(ErrorMessage = "Role is required.")]
```

```
[StringLength(100, MinimumLength = 2)]
```

```
public string Role { get; set; } = string.Empty;
```

What's happening:

Required prevents missing values, while StringLength requires between **2 and 100 characters**.

Why it matters:

It prevents obviously invalid application records such as an empty company or a one-character role.

Byte 4: Application Status

Builds on: Byte 1

In plain terms:

Status tracks where the application currently is in the hiring process. It defaults to "Applied".

The code:

```
[Required(ErrorMessage = "Status is required.")]
```

```
public string Status { get; set; } = "Applied";
```

What's happening:

If no status is explicitly supplied when an object is created, it starts as Applied.

Why it matters:

The status is important for tracking progress, but the Required attribute alone doesn't guarantee that the value is one of the application's valid statuses. That is handled separately.

Byte 5: Defining Allowed Statuses

Builds on: Byte 4

In plain terms:

The model defines the exact set of statuses that an application is allowed to have.

The code:

```
public static readonly string[] AllowedStatuses =  
    new[]  
    {  
        "Wishlist",  
        "Applied",  
        "Interview",  
        "Offer",  
        "Rejected"  
    };
```

What's happening:

This array acts as the model's list of valid status values.

Why it matters:

It prevents the application from accepting arbitrary status values such as "Maybe" or "Cancelled" unless they are deliberately added to the allowed list.

Byte 6: Optional Application Date

Builds on: Byte 4

In plain terms:

AppliedOn stores the date the application was submitted. The ? means it can be empty.

The code:

```
[DataType(DataType.Date)]  
[Display(Name = "Applied On")]  
public DateTime? AppliedOn { get; set; }
```

What's happening:

DateTime? means the property can contain either a date or null. The attributes mainly provide metadata for validation/UI representation.

Why it matters:

A wishlist item hasn't necessarily been applied to yet, so it makes sense for its application date to be empty.

Byte 7: Optional Job URL

Builds on: Byte 1

In plain terms:

JobUrl stores the link to the job posting. It is optional, but if provided, it should be a valid URL.

The code:

```
[Url(ErrorMessage =  
    "Please enter a valid URL (e.g. https://example.com/job).")]  
[Display(Name = "Job URL")]  
public string? JobUrl { get; set; }
```

What's happening:

string? allows the field to be null, while [Url] provides URL-format validation when a value is supplied.

Why it matters:

Users don't have to provide a job link, but invalid links can be rejected during validation.

Byte 8: Optional Notes

Builds on: Byte 1

In plain terms:

Notes allows extra information about an application, such as interview details or recruiter notes.

The code:

```
[StringLength(1000,  
    ErrorMessage = "Notes cannot be longer than 1000 characters.")]  
public string? Notes { get; set; }
```

What's happening:

The field is optional, but its content cannot exceed **1000 characters**.

Why it matters:

It provides flexibility for extra information while preventing excessively large values.

Byte 9: Recording When the Application Was Created

Builds on: Byte 1

In plain terms:

CreatedAt records when the application record was originally created.

The code:

```
[Display(Name = "Created At")]
```

```
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

What's happening:

When a new Application object is created, CreatedAt automatically receives the current UTC time.

Why it matters:

The system can know when an application was added, which is useful for sorting, tracking, and auditing records.

Byte 10: Custom Validation

Builds on: Bytes 5–9

In plain terms:

Some rules cannot be expressed by a single attribute because they depend on **multiple properties**. Validate() handles those business rules.

The code:

```
public IEnumerable<ValidationResult> Validate(
    ValidationContext validationContext)
{
    // custom validation rules
}
```

What's happening:

Because the class implements IValidatableObject, the validation system can call this method and collect any ValidationResult objects it returns.

Why it matters:

This is where rules such as "**AppliedOn is required unless the status is Wishlist**" can be enforced.

Byte 11: Validating the Status Value

Builds on: Bytes 5 and 10

In plain terms:

The custom validation checks whether the status actually belongs to the allowed list.

The code:

```

if (!AllowedStatuses.Contains(Status))
{
    yield return new ValidationResult(
        "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected.",
        new[] { nameof(Status) });
}

```

What's happening:

Contains() searches the allowed-status array. If the supplied status isn't found, a validation error is produced and associated with the Status field.

Why it matters:

[Required] only checks that something was supplied. This additional rule checks that the supplied value is actually valid.

Byte 12: Rejecting Blank Company and Role Values

Builds on: Bytes 3 and 10

In plain terms:

The model performs an additional check to catch company or role values that are technically strings but contain no meaningful content.

The code:

```

if (string.IsNullOrEmpty(Company) ||
    Company.Trim().Length < 2)
{
    yield return new ValidationResult(
        "Company must be 2-100 characters.",
        new[] { nameof(Company) });
}

```

What's happening:

Values such as " " are rejected. The same validation pattern is used for Role.

Why it matters:

This provides stronger protection than simply checking whether the property exists as a string.

Byte 13: The Wishlist Date Rule

Builds on: Bytes 4, 6, and 10

In plain terms:

The model has a conditional business rule: **Wishlist applications don't need an applied date, but other statuses do.**

The code:

```
if (Status != "Wishlist" && AppliedOn is null)
{
    yield return new ValidationResult(
        "Applied date is required unless status is Wishlist.",
        new[] { nameof(AppliedOn) });
}
```

What's happening:

If the status is anything other than Wishlist and there is no AppliedOn date, validation fails.

For example:

Wishlist + no date → Valid

Applied + no date → Invalid

Interview + date → Valid

Why it matters:

This ensures the stored application data matches the workflow rules of the placement tracker.

1. obj folder

obj means **Object**. It is a temporary/build-support folder created by .NET.

It contains files that .NET uses while **building, restoring packages, and compiling** your project.

You normally **do not edit anything inside obj manually**.

2. obj/Debug

This contains intermediate files created when you build the project in **Debug mode**.

In simple terms:

.cs files → compilation/intermediate processing → obj/Debug → final output in bin

It helps .NET keep track of the current build.

3. PlacementTracker.Api.csproj.nuget.dgspec.json

This is a **generated NuGet dependency specification file**.

It contains information about:

- Your project
- NuGet packages
- Target framework
- Package dependencies
- How packages should be restored

You generally **don't edit it manually**.

4. PlacementTracker.Api.csproj.nuget.g.props

This is another **generated NuGet/MSBuild file**.

It contains automatically generated build properties related to your NuGet packages and project configuration.

Think of it as:

"Extra build settings that NuGet gives to the .NET build system."

5. PlacementTracker.Api.csproj.nuget.g.targets

Similar to the .props file, but contains generated **MSBuild targets/tasks** used during the build process.

You normally don't touch this file either.

6. project.assets.json

This is an important generated file for **NuGet package dependency information**.

It records which packages your project depends on and where their required libraries come from.

For example, your project uses things like:

Microsoft.EntityFrameworkCore

Npgsql

ASP.NET Core

project.assets.json helps .NET know what packages and dependencies are available during compilation.

7. project.nuget.cache

This is a **NuGet restore cache file**.

It helps .NET/NuGet remember information about the previous package restoration process so it doesn't have to unnecessarily repeat work.

8. Properties

This is different from obj because **you may actually work with files inside it**.

In your ASP.NET Core project, it commonly contains:

Properties/

launchSettings.json

launchSettings.json controls development-time settings such as:

- How the application launches
- Development URLs/ports
- Launch profiles
- Environment variables

For example, it can determine whether you launch the API using HTTP or HTTPS during development.

9. appsettings.Development.json

This contains **configuration specifically for the Development environment**.

For example:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information"
    }
  }
}
```

When your application runs in Development mode, these settings can override or supplement the normal appsettings.json.

10. appsettings.json

This is the application's **main configuration file**.

It can contain things like:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "..."
  }
}
```

In your project, this is particularly important for configuration such as the **PostgreSQL connection string**.

11. PlacementTracker.Api.csproj

This is one of the **most important files in the project structure**.

It defines information about your .NET project, such as:

- Target .NET version
- NuGet package references
- Project settings
- Build configuration

For example:

```
<TargetFramework>net8.0</TargetFramework>
```

and dependencies such as:

```
<PackageReference Include="Microsoft.EntityFrameworkCore" />
```

Think of it as the **project's configuration/definition file**.

Very short way to remember it

Item	Main purpose
obj	Temporary/intermediate build files
Debug	Debug build intermediates
.nuget.dgspec.json	NuGet dependency specification
.nuget.g.props	Generated NuGet build properties

Item	Main purpose
.nuget.g.targets	Generated NuGet build targets
project.assets.json	Resolved package/dependency information
project.nuget.cache	NuGet restore cache
Properties	Development/launch configuration
appsettings.Development.json	Development settings
appsettings.json	Main application configuration
.csproj	Defines the .NET project and its dependencies

Program.cs

Byte 1: What Program.cs does

Builds on: None — starting point

In plain terms:

Program.cs is the **startup and configuration file** for your ASP.NET Core API. It sets up services, database access, CORS, Swagger, middleware, routes, and finally starts the application.

The code:

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Configure services...
```

```
var app = builder.Build();
```

```
// Configure application...
```

```
app.Run();
```

What's happening:

The file has two major stages:

1. **Configure services** using builder
2. **Build and configure the running application** using app

Why it matters:

Without Program.cs, ASP.NET Core wouldn't know how your API should be configured or started.

Byte 2: Handling PostgreSQL DateTime Behavior

Builds on: Byte 1

In plain terms:

Your application uses both normal calendar dates and UTC timestamps. This setting tells Npgsql to use its legacy timestamp behavior so different DateTime kinds don't cause conflicts.

The code:

```
AppContext.SetSwitch(  
    "Npgsql.EnableLegacyTimestampBehavior",  
    true);
```

What's happening:

Npgsql is the PostgreSQL provider used by EF Core. This switch changes how it handles C# DateTime values, allowing the application to work with the mixed date/time usage in this project.

Why it matters:

Without this setting, certain DateTime values—especially Unspecified versus Utc—could cause PostgreSQL/Npgsql timestamp errors.

Byte 3: Creating the Application Builder

Builds on: Byte 1

In plain terms:

WebApplication.CreateBuilder() creates the object used to configure the API before it starts running.

The code:

```
var builder = WebApplication.CreateBuilder(args);
```

What's happening:

The builder loads things such as configuration, environment information, logging, and the service container.

Why it matters:

Almost all the configuration that follows—database, controllers, Swagger, and CORS—is registered through builder.

Byte 4: Registering API Services

Builds on: Byte 3

In plain terms:

These lines tell ASP.NET Core that this project is a **Web API** and enable API documentation through Swagger.

The code:

```
builder.Services.AddControllers();  
  
builder.Services.AddEndpointsApiExplorer();  
  
builder.Services.AddSwaggerGen();
```

What's happening:

AddControllers() enables controller-based API endpoints. AddSwaggerGen() generates Swagger/OpenAPI documentation, which allows you to view and test endpoints through a browser during development.

Why it matters:

Without AddControllers(), your ApplicationController wouldn't be registered as an API controller.

Byte 5: Getting the PostgreSQL Connection String

Builds on: Byte 3

In plain terms:

The application needs a **connection string**—the information required to connect to PostgreSQL. This code looks for it in configuration before using a local fallback.

The code:

```
var connectionString =  
    builder.Configuration.GetConnectionString("Default")  
    ?? Environment.GetEnvironmentVariable(  
        "ConnectionStrings__Default")  
    ??  
    "Host=127.0.0.1;Port=5432;Database=placement_tracker;Username=postgres;Password=postgres";
```

What's happening:

The ?? operator means **"use the next value if the previous one is null."** The application therefore tries configuration first, then an environment variable, then the fallback connection string.

Why it matters:

It allows the same application to work in different environments without hardcoding the database configuration everywhere.

Byte 6: Connecting EF Core to PostgreSQL

Builds on: Bytes 2 and 5

In plain terms:

This registers your ApplicationDbContext and tells Entity Framework Core to use **PostgreSQL**.

The code:

```
builder.Services.AddDbContext<ApplicationDbContext>(
```

```
options => options.UseNpgsql(connectionString));
```

What's happening:

ApplicationDbContext is the database bridge you examined earlier. UseNpgsql() tells EF Core that PostgreSQL is the database provider and gives it the connection string.

Why it matters:

This is what enables the controller's `_db.Applications` operations to ultimately communicate with PostgreSQL.

Byte 7: Defining the CORS Policy

Builds on: Byte 3

In plain terms:

Your Vue frontend and .NET API run on different development origins. **CORS** (Cross-Origin Resource Sharing) controls whether the browser is allowed to let the frontend call the API.

The code:

```
const string VueDevPolicy = "VueDev";

builder.Services.AddCors(options =>
{
    options.AddPolicy(VueDevPolicy, policy =>
    {
        policy.WithOrigins("http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

What's happening:

Only requests coming from the Vue development server at `http://localhost:5173` are allowed. Any HTTP method and headers are permitted.

Why it matters:

Without the correct CORS policy, the browser could block your Vue frontend from calling the API even though the API itself is working.

Byte 8: Building the Application

Builds on: Bytes 3–7

In plain terms:

After all services have been registered, `builder.Build()` creates the actual ASP.NET Core application.

The code:

```
var app = builder.Build();
```

What's happening:

The configuration phase ends here. `app` is now the object used to configure how incoming requests should flow through the application.

Why it matters:

Everything after this point mainly configures the **request pipeline and application startup behavior**.

Byte 9: Enabling Swagger Only in Development

Builds on: Byte 8

In plain terms:

Swagger is enabled only when the application is running in the Development environment.

The code:

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

What's happening:

When running locally in Development, Swagger provides an interactive API page where you can inspect and test endpoints.

Why it matters:

It gives you a convenient development/testing tool without automatically exposing the Swagger UI in other environments.

Byte 10: Configuring the Request Pipeline

Builds on: Bytes 7–9

In plain terms:

These lines define important processing steps that requests go through before reaching your controllers.

The code:

```
app.UseCors(VueDevPolicy);
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

What's happening:

- UseCors() applies your Vue CORS policy.
- UseAuthorization() enables authorization middleware.
- MapControllers() connects HTTP routes to controller actions.

For example:

```
GET /api/applications
```

↓

```
ApplicationsController.GetAll()
```

Why it matters:

Without MapControllers(), ASP.NET Core wouldn't know how to route requests to your ApplicationsController.

Byte 11: Skipping Database Initialization During Tests

Builds on: Byte 8

In plain terms:

The application creates its database tables during normal startup, but deliberately skips this step when running tests.

The code:

```
if (!app.Environment.IsEnvironment("Testing"))  
{  
    // database initialization  
}
```

What's happening:

The application checks whether the current environment is "Testing". If it is, this entire database initialization block is skipped.

Why it matters:

Your automated tests use isolated test database providers. Skipping this block prevents tests from accidentally modifying or depending on your real PostgreSQL database.

Byte 12: Creating a Database Scope

Builds on: Byte 11

In plain terms:

Inside the startup database setup, the application creates a temporary **dependency-injection scope** so it can safely obtain the database context.

The code:

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider
        .GetRequiredService<ApplicationDbContext>();
}
```

What's happening:

The application asks its dependency-injection container for an ApplicationDbContext. The using block ensures the scope is properly disposed afterward.

Why it matters:

This provides a correctly managed DbContext during application startup rather than manually constructing one.

Byte 13: Creating Tables If Needed

Builds on: Byte 12

In plain terms:

EnsureCreated() checks whether the database exists in the expected form and creates the database schema if necessary.

The code:

```
db.Database.EnsureCreated();
```

What's happening:

For this project, startup can create the required tables if they don't already exist.

Why it matters:

It makes local setup easier because the application doesn't require a separate database-creation step just to start working.

Byte 14: Adding Development Sample Data

Builds on: Byte 13

In plain terms:

When running in Development, the application adds three sample applications—but only if the table is currently empty.

The code:

```
if (app.Environment.IsDevelopment() &&  
    !db.Applications.Any())  
{  
    db.Applications.AddRange(  
        // 3 sample Application objects  
    );  
  
    db.SaveChanges();  
}
```

What's happening:

The two conditions mean:

Development environment

AND

Applications table is empty

↓

Add sample records

AddRange() adds multiple records, and SaveChanges() writes them to PostgreSQL.

Why it matters:

It gives you realistic data immediately when developing the frontend, while avoiding duplicate sample records every time the API starts.

Byte 15: Starting the API

Builds on: Bytes 8–14

In plain terms:

app.Run() starts the web server and keeps the API running so it can receive HTTP requests.

The code:

```
app.Run();
```

What's happening:

Once this line executes, ASP.NET Core begins listening for incoming requests according to the configured server settings.

Why it matters:

Without `app.Run()`, the application would finish executing instead of staying alive as a web API.

Byte 16: Making Program Accessible to Integration Tests

Builds on: Byte 15

In plain terms:

The final declaration makes the automatically generated `Program` class accessible to your integration tests.

The code:

```
public partial class Program { }
```

What's happening:

`WebApplicationFactory<Program>` in your integration tests needs access to the `Program` type so it can create a test version of the application.

Why it matters:

Without this, your integration tests may not be able to locate and start the API using `WebApplicationFactory`.

- **tests/bin** → Contains the **compiled output of your tests** — DLLs and other files needed to actually run the test suite.
- **tests/obj** → Contains **intermediate build files** created while compiling/restoring the test project.

Simple way to remember:

tests/obj = temporary build/compilation files

tests/bin = final compiled test files used to run tests

ApplicationsApiTests.cs

Byte 1: What This Test Class Is For

Builds on: None — starting point

In plain terms:

ApplicationsApiTests tests the backend API controller without starting the real application or connecting to PostgreSQL. It checks whether operations like GET, POST, PUT, and DELETE return the correct results and HTTP responses.

The code:

```
public class ApplicationsApiTests
{
    // API controller tests with isolated InMemory EF provider.
    // Never touches PostgreSQL.
}
```

What's happening:

This class groups together tests for ApplicationController. The tests use an isolated EF Core InMemory database, so each test can safely create its own data.

Why it matters:

The tests can run quickly and repeatedly without changing the real PostgreSQL database.

Byte 2: Creating a Controller with a Test Database

Builds on: Byte 1

In plain terms:

The helper method creates an ApplicationController and gives it a test database. It also provides the controller with a fake HTTP context so it behaves like a controller handling an HTTP request.

The code:

```
private static ApplicationController ControllerWithDb(
    Data.ApplicationDbContext db)
{
    var controller = new ApplicationController(db);

    controller.ControllerContext = new ControllerContext
    {
```

```

        HttpContext = new DefaultHttpContext()
    };

    return controller;
}

```

What's happening:

The real controller normally receives its `ApplicationDbContext` through dependency injection. Here, the test creates the controller manually and supplies the test database.

`DefaultHttpContext` provides a lightweight fake HTTP environment.

Why it matters:

Every test gets a properly configured controller without starting the entire web server.

Byte 3: Testing GET All with Data

Builds on: Byte 2

In plain terms:

This test verifies that `GetAll()` returns all applications currently stored in the database.

The code:

```

db.Applications.Add(TestHelpers.ValidApplication());
db.Applications.Add(new Models.Application
{
    Company = "Northstar",
    Role = "Graduate Engineer",
    Status = "Interview",
    AppliedOn = new DateTime(2026, 9, 12),
    CreatedAt = DateTime.UtcNow
});

await db.SaveChangesAsync();

var controller = ControllerWithDb(db);
var result = await controller.GetAll();

```

What's happening:

Two applications are inserted into the test database. The controller's GetAll() method is then called directly.

The test later verifies:

```
var ok = Assert.IsType<OkObjectResult>(result.Result);  
var list = Assert.IsAssignableFrom<List<ApplicationResponseDto>>(ok.Value);  
  
Assert.Equal(2, list.Count);
```

So the response must be 200 OK and contain two DTOs.

Why it matters:

It confirms both the data retrieval and the expected API response type.

Byte 4: Testing an Empty Database

Builds on: Byte 3

In plain terms:

An API should also behave correctly when there is no data. This test checks that GetAll() returns an empty list rather than failing.

The code:

```
using var db = TestHelpers.CreateDb();  
  
var controller = ControllerWithDb(db);  
var result = await controller.GetAll();  
  
var ok = Assert.IsType<OkObjectResult>(result.Result);  
var list = Assert.IsAssignableFrom<List<ApplicationResponseDto>>(ok.Value);  
  
Assert.Empty(list);
```

What's happening:

No applications are inserted. GetAll() is called and the result must still be 200 OK, with an empty list.

Why it matters:

The dashboard can safely handle a new installation where no applications exist yet.

Byte 5: Testing GET One for an Existing Record

Builds on: Byte 3

In plain terms:

This test checks that requesting a specific application ID returns the correct application and its response DTO.

The code:

```
db.Applications.Add(TestHelpers.ValidApplication());  
await db.SaveChangesAsync();
```

```
var id = db.Applications.Single().Id;
```

```
var controller = ControllerWithDb(db);  
var result = await controller.GetOne(id);
```

What's happening:

The test first saves an application, obtains its generated database ID, and passes that ID to `GetOne()`.

It then verifies:

```
var ok = Assert.IsType<OkObjectResult>(result.Result);  
var dto = Assert.IsType<ApplicationResponseDto>(ok.Value);
```

```
Assert.Equal("Acme Labs", dto.Company);
```

```
Assert.Equal("Backend Intern", dto.Role);
```

Why it matters:

It confirms that the API retrieves the requested record rather than simply returning some application.

Byte 6: Testing a Missing Record

Builds on: Byte 5

In plain terms:

If a requested application doesn't exist, the API should return **404 Not Found** instead of pretending the record exists.

The code:

```
var controller = ControllerWithDb(db);
```

```
var result = await controller.GetOne(999999);
```

```
Assert.IsType<NotFoundObjectResult>(result.Result);
```

What's happening:

The test asks for an ID that doesn't exist. The controller is expected to return a `NotFoundObjectResult`.

Why it matters:

This verifies the API's error handling for invalid or nonexistent resource IDs.

Byte 7: Testing Successful POST/Create

Builds on: Byte 2

In plain terms:

This test verifies that a valid application can be created and saved to the database.

The code:

```
var controller = ControllerWithDb(db);
```

```
var result = await controller.Create(  
    TestHelpers.ValidCreateDto());
```

```
var created = Assert.IsType<CreatedAtActionResult>(  
    result.Result);
```

```
Assert.Equal(201, created.StatusCode);
```

```
Assert.Equal(1, db.Applications.Count());
```

What's happening:

A valid DTO is sent to `Create()`. The expected response is `CreatedAtActionResult`, representing **HTTP 201 Created**.

The database is also checked to make sure exactly one record was saved.

Why it matters:

It verifies both sides of creation: the correct HTTP response and actual persistence.

Byte 8: Testing Invalid POST/Create

Builds on: Byte 7

In plain terms:

This test checks that invalid input is rejected and does not create a database record.

The code:

```
controller.ModelState.AddModelError(  
    "Company",  
    "Company is required."  
);
```

```
var dto = TestHelpers.ValidCreateDto();  
dto.Company = "";
```

```
var result = await controller.Create(dto);
```

The assertions are:

```
var badRequest =  
    Assert.IsAssignableFrom<ObjectResult>(result.Result);
```

```
Assert.IsType<ValidationProblemDetails>(  
    badRequest.Value);
```

```
Assert.Equal(0, db.Applications.Count());
```

What's happening:

The test places a validation error into ModelState and sends an invalid DTO. The controller should return validation information instead of saving it.

Why it matters:

Bad input must not reach the database. This protects the application's data integrity.

Byte 9: Testing Successful PUT/Update

Builds on: Byte 7

In plain terms:

This test verifies that an existing application can be updated and that the changed values are actually saved.

The code:

```
var id = db.Applications.Single().Id;

var controller = ControllerWithDb(db);
var result = await controller.Update(
    id,
    TestHelpers.ValidUpdateDto()
);
```

The test checks:

```
Assert.Equal("Contoso", dto.Company);
```

```
var saved = db.Applications.Single();
```

```
Assert.Equal("Contoso", saved.Company);
```

```
Assert.Equal("Offer Role", saved.Role);
```

```
Assert.Equal("Offer", saved.Status);
```

What's happening:

The test updates an existing record, checks the response DTO, and then checks the database record itself.

Why it matters:

Checking both response and stored data proves that the update wasn't merely reflected in the response—it was actually persisted.

Byte 10: Testing PUT with a Missing ID

Builds on: Byte 6 and Byte 9

In plain terms:

Updating an ID that doesn't exist should produce **404 Not Found**.

The code:

```
var result = await controller.Update(
    999999,
    TestHelpers.ValidUpdateDto()
);
```

```
Assert.IsType<NotFoundObjectResult>(result.Result);
```

What's happening:

The test sends valid update data, but the target ID does not exist.

Why it matters:

The API must distinguish between "invalid data" and "resource doesn't exist."

Byte 11: Testing Invalid PUT Data

Builds on: Byte 8 and Byte 9

In plain terms:

An existing record can still receive invalid update data. This test ensures validation catches it.

The code:

```
controller.ModelState.AddModelError(  
    "Status",  
    "Invalid status."  
);
```

```
var dto = TestHelpers.ValidUpdateDto();  
dto.Status = "Hired";
```

```
var result = await controller.Update(id, dto);
```

What's happening:

The test gives the update an invalid status and adds a validation error to ModelState. It then expects validation-problem output.

Why it matters:

Update operations need the same validation protection as creation operations.

Byte 12: Testing Successful DELETE

Builds on: Byte 6 and Byte 9

In plain terms:

This test verifies that an existing application can be deleted and that the record disappears from the database.

The code:

```
var id = db.Applications.Single().Id;
```

```
var controller = ControllerWithDb(db);
```

```
var result = await controller.Delete(id);
```

```
Assert.IsType<NoContentResult>(result);
```

```
Assert.Equal(0, db.Applications.Count());
```

What's happening:

The existing application's ID is passed to Delete(). The expected response is 204 No Content, and the database must contain zero records afterward.

Why it matters:

It proves that deletion actually happens and that the API returns the appropriate REST response.

Byte 13: Testing DELETE with a Missing ID

Builds on: Byte 12

In plain terms:

Deleting an application that doesn't exist should return **404 Not Found** rather than succeeding silently.

The code:

```
var controller = ControllerWithDb(db);
```

```
var result = await controller.Delete(999999);
```

```
Assert.IsType<NotFoundObjectResult>(result);
```

What's happening:

The test sends a nonexistent ID to the delete operation and verifies the controller reports that the resource was not found.

Why it matters:

This makes DELETE behavior consistent with GET and PUT when the requested resource doesn't exist.

IntegrationTests.cs

Byte 1: Purpose of the Integration Tests

Builds on: None — starting point

In plain terms:

IntegrationTests checks the API as a whole through actual HTTP requests. Unlike controller unit tests, these tests go through routing, middleware, model binding, controllers, and the database configuration.

The code:

```
public class IntegrationTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;
}
```

What's happening:

WebApplicationFactory<Program> creates a test version of your ASP.NET Core application. The tests can then create an HttpClient and call endpoints such as /api/applications.

Why it matters:

This verifies that the different backend pieces work together, not just that individual controller methods work.

Byte 2: Creating a Separate Test Database

Builds on: Byte 1

In plain terms:

The tests must not modify your real PostgreSQL database. A unique InMemory database is created specifically for the integration-test application.

The code:

```
var dbName = "IntegrationTests-" + Guid.NewGuid();

services.AddDbContext<ApplicationDbContext>(options =>
    options.UseInMemoryDatabase(dbName));
```

What's happening:

Guid.NewGuid() creates a unique identifier, so the test database gets a unique name. UseInMemoryDatabase() tells EF Core to store test data in memory instead of PostgreSQL.

Why it matters:

Your integration tests can create, update, and delete records without affecting your real placement_tracker PostgreSQL database.

Byte 3: Running the Application in Testing Environment

Builds on: Byte 2

In plain terms:

The test application is explicitly started with the Testing environment. Your Program.cs can use this environment to avoid production/development database behavior.

The code:

```
builder.UseEnvironment("Testing");
```

What's happening:

ASP.NET Core has different environments such as Development, Testing, and Production. Here, the application is told that these requests are coming from the testing environment.

Why it matters:

According to the comment in this code, your Program.cs uses this environment so the real PostgreSQL database isn't touched during these tests.

Byte 4: Replacing the Real DbContext

Builds on: Byte 2 and Byte 3

In plain terms:

The test removes the application's normal ApplicationDbContext configuration and replaces it with the InMemory version.

The code:

```
var descriptor = services.SingleOrDefault(
    d => d.ServiceType == typeof(DbContextOptions<ApplicationDbContext>));

if (descriptor != null)
{
    services.Remove(descriptor);
}

services.AddDbContext<ApplicationDbContext>(options =>
    options.UseInMemoryDatabase(dbName));
```

What's happening:

The application's existing EF Core database registration is found and removed. Then a new registration points `ApplicationDbContext` to the test's InMemory database.

Why it matters:

Without replacing the original configuration, the integration tests could accidentally use PostgreSQL.

Byte 5: Why the Database Name Is Created Outside `ConfigureServices`

Builds on: Byte 4

In plain terms:

The unique database name is deliberately created before `ConfigureServices` runs. This ensures different executions of the configuration delegate refer to the same test database.

The code:

```
var dbName = "IntegrationTests-" + Guid.NewGuid();

_factory = factory.WithWebHostBuilder(builder =>
{
    builder.ConfigureServices(services =>
    {
        services.AddDbContext<ApplicationDbContext>(options =>
            options.UseInMemoryDatabase(dbName));
    });
});
```

What's happening:

If `Guid.NewGuid()` were called inside `ConfigureServices`, multiple executions could produce different names. By generating it once outside, all configuration executions use the same `dbName`.

Why it matters:

This prevents confusing test behavior where different parts of the test application accidentally point to different InMemory stores.

Byte 6: Creating an HTTP Client

Builds on: Byte 1

In plain terms:

CreateClient() gives the test an HTTP client that communicates with the test version of the ASP.NET Core application.

The code:

```
var client = _factory.CreateClient();
```

```
var response = await client.GetAsync(  
    "/api/applications");
```

What's happening:

Instead of directly calling:

```
controller.GetAll()
```

the test makes a real HTTP GET request to the API endpoint.

Why it matters:

This tests the API the way your Vue frontend actually uses it.

Byte 7: Checking the HTTP Status Code

Builds on: Byte 6

In plain terms:

Integration tests verify the actual HTTP response, not just a C# result object.

The code:

```
Assert.Equal(  
    HttpStatusCode.OK,  
    response.StatusCode);
```

What's happening:

HttpStatusCode.OK represents HTTP **200 OK**. The test checks that /api/applications responds with that status.

Why it matters:

It confirms the API's externally visible behavior, including its HTTP status codes.

Byte 8: Reading JSON from the API

Builds on: Byte 6 and Byte 7

In plain terms:

Your API returns JSON. ReadFromJsonAsync converts that JSON response into a C# object that the test can inspect.

The code:

```
var list =  
    await response.Content  
        .ReadFromJsonAsync<  
            List<DTOs.ApplicationResponseDto>>();
```

```
Assert.NotNull(list);
```

What's happening:

The HTTP response body is JSON, and .NET's JSON helper deserializes it into a `List<ApplicationResponseDto>`.

Why it matters:

This verifies not only that the endpoint returned 200, but also that its response can actually be consumed as the expected JSON structure.

Byte 9: Testing a GET for a Missing Application

Builds on: Byte 6 and Byte 7

In plain terms:

The integration test also checks the API's error behavior when a requested application doesn't exist.

The code:

```
var response = await client.GetAsync(  
    "/api/applications/999999");
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    response.StatusCode);
```

What's happening:

The test makes a real HTTP request for an ID that doesn't exist. The API should respond with **404 Not Found**.

Why it matters:

It confirms that the controller's missing-resource behavior works correctly through the complete HTTP pipeline.

Byte 10: Testing POST Through HTTP

Builds on: Byte 6 and Byte 8

In plain terms:

This test sends JSON to the real API endpoint to verify that creating an application works from an HTTP client's perspective.

The code:

```
var post = await client.PostAsJsonAsync(
    "/api/applications",
    new
    {
        company = "Acme Labs",
        role = "Backend Intern",
        status = "Applied",
        appliedOn = "2026-09-16"
    });
```

```
Assert.Equal(
    HttpStatusCode.Created,
    post.StatusCode);
```

What's happening:

PostAsJsonAsync converts the anonymous C# object into JSON and sends it as an HTTP POST request.

The expected response is **201 Created**.

Why it matters:

This tests JSON serialization, HTTP routing, model binding, controller logic, and database saving together.

Byte 11: Reading the Created Application

Builds on: Byte 10

In plain terms:

After creation, the test reads the JSON returned by the API and verifies that the created application's data is correct.

The code:

```
var created =
```

```
await post.Content
    .ReadFromJsonAsync<
        DTOs.ApplicationResponseDto>());
```

```
Assert.NotNull(created);
```

```
Assert.Equal("Acme Labs", created!.Company);
```

What's happening:

The 201 Created response contains an ApplicationResponseDto. The test deserializes the JSON and checks the returned company.

Why it matters:

It confirms the API doesn't just say creation succeeded—the returned resource also contains the expected data.

Byte 12: Full CRUD Roundtrip

Builds on: Bytes 6–11

In plain terms:

This is the most complete test in the file. It creates an application, reads it, updates it, deletes it, and finally confirms it is gone.

The code:

```
POST /api/applications
GET /api/applications/{id}
PUT /api/applications/{id}
DELETE /api/applications/{id}
GET /api/applications/{id}
```

What's happening:

The test follows the complete lifecycle of one application:

1. POST → **201 Created**
2. GET → **200 OK**
3. PUT → **200 OK**
4. DELETE → **204 No Content**
5. GET again → **404 Not Found**

Why it matters:

It verifies that the complete CRUD workflow works correctly over HTTP.

Byte 13: Testing PUT/Update

Builds on: Byte 12

In plain terms:

The PUT request sends replacement data for an existing application and expects the API to confirm the update.

The code:

```
var put = await client.PutAsJsonAsync(
    $"/api/applications/{created.Id}",
    new
    {
        company = "Contoso",
        role = "Offer Role",
        status = "Offer",
        appliedOn = "2026-09-05"
    });
```

```
Assert.Equal(
    HttpStatusCode.OK,
    put.StatusCode);
```

What's happening:

The created application's ID is inserted into the URL. The updated fields are sent as JSON.

Why it matters:

It confirms that the frontend can send JSON updates to a specific application through the API.

Byte 14: Testing DELETE

Builds on: Byte 12

In plain terms:

The test verifies that an existing application can be removed through the HTTP DELETE endpoint.

The code:

```
var delete = await client.DeleteAsync(
```

```
$/api/applications/{created.Id}");
```

```
Assert.Equal(  
    HttpStatusCode.NoContent,  
    delete.StatusCode);
```

What's happening:

The test sends DELETE to the application's URL. The expected HTTP response is **204 No Content**, meaning the operation succeeded without requiring a response body.

Why it matters:

It confirms the public DELETE API works correctly rather than only testing the controller method internally.

Byte 15: Confirming DELETE Actually Worked

Builds on: Byte 14

In plain terms:

A successful DELETE response isn't enough. The test performs another GET to prove that the application is actually gone.

The code:

```
var getAfter = await client.GetAsync(  
    $"/api/applications/{created.Id}");
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    getAfter.StatusCode);
```

What's happening:

The same ID is requested after deletion. Since the record was deleted, the API should now return **404 Not Found**.

Why it matters:

This verifies the complete state change rather than trusting only the DELETE response.

Byte 16: Testing Invalid POST Data

Builds on: Byte 10

In plain terms:

This test checks that invalid application data is rejected at the HTTP API level.

The code:

```
var response = await client.PostAsJsonAsync(
    "/api/applications",
    new
    {
        company = "",
        role = "x",
        status = "Hired",
        appliedOn = (string?)null
    });
```

```
Assert.Equal(
    HttpStatusCode.BadRequest,
    response.StatusCode);
```

What's happening:

The request deliberately contains invalid values: an empty company, a short role, an invalid status, and no applied date.

The API is expected to return **400 Bad Request**.

Why it matters:

This confirms validation works through the real HTTP pipeline, not just when the controller is called directly.

Byte 17: Testing PUT with a Nonexistent ID

Builds on: Byte 13

In plain terms:

Even valid update data cannot update an application that doesn't exist.

The code:

```
var response = await client.PutAsJsonAsync(
    "/api/applications/999999",
    new
    {
        company = "Contoso",
```

```
    role = "Backend Intern",  
    status = "Applied",  
    appliedOn = "2026-09-05"  
});
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    response.StatusCode);
```

What's happening:

The data is valid, but the target ID does not exist. Therefore, the API returns **404 Not Found**.

Why it matters:

It verifies that resource existence is checked separately from input validation.

Byte 18: Testing DELETE with a Nonexistent ID

Builds on: Byte 14

In plain terms:

The final edge case checks that deleting an ID that doesn't exist produces a proper 404 response.

The code:

```
var response = await client.DeleteAsync(  
    "/api/applications/999999");
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    response.StatusCode);
```

What's happening:

The API receives a DELETE request for a nonexistent resource and reports that the resource cannot be found.

Why it matters:

This keeps DELETE behavior consistent with GET and PUT for missing records.

TestHelpers.cs

Byte 1: Purpose of TestHelpers

Builds on: None — starting point

In plain terms:

TestHelpers is a collection of reusable helper methods for the test suite. Instead of repeating database setup and sample application data in every test, the tests can call these helpers.

The code:

```
public static class TestHelpers
{
    public static ApplicationDbContext CreateDb()
    {
        // ...
    }
}
```

What's happening:

The class is static, meaning it doesn't need to be instantiated. Its methods can be called directly, for example:

```
var db = TestHelpers.CreateDb();
```

Why it matters:

It keeps the actual tests shorter and prevents the same setup code from being copied into many test methods.

Byte 2: Building an EF Core Test Database

Builds on: Byte 1

In plain terms:

CreateDb() creates an Entity Framework Core DbContext configured specifically for testing.

The code:

```
var options =
    new DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(
            databaseName: Guid.NewGuid().ToString())
        .Options;
```

What's happening:

DbContextOptionsBuilder configures how ApplicationDbContext should connect to a database.

Instead of calling UseNpgsql() for PostgreSQL, the test uses UseInMemoryDatabase().

Why it matters:

The tests can perform database operations without connecting to your real PostgreSQL server.

Byte 3: Giving Each Test a Unique Database

Builds on: Byte 2

In plain terms:

The test database gets a randomly generated name using Guid.NewGuid(). This gives each call to CreateDb() its own isolated database.

The code:

```
.UseInMemoryDatabase(  
    databaseName: Guid.NewGuid().ToString())
```

What's happening:

A GUID is a practically unique identifier such as:

8f1c...-...

That value becomes the InMemory database name.

Why it matters:

Data created by one test won't accidentally appear in another test. Each test starts with a clean database.

Byte 4: Creating the ApplicationDbContext

Builds on: Byte 2

In plain terms:

Once the database options are ready, they are used to create the application's EF Core DbContext.

The code:

```
var db = new ApplicationDbContext(options);
```

What's happening:

ApplicationDbContext is the EF Core class that represents the application's database session and exposes things such as:

db.Applications

The important point is that this particular context is configured to use the InMemory provider.

Why it matters:

The tests can now use normal EF Core operations such as `Add()`, `Count()`, `Single()`, and `SaveChangesAsync()` without PostgreSQL.

Byte 5: Ensuring the Test Database Exists

Builds on: Byte 4

In plain terms:

`EnsureCreated()` makes sure the InMemory database is ready before the test starts using it.

The code:

```
db.Database.EnsureCreated();  
  
return db;
```

What's happening:

The context's database API is asked to create the database if necessary. The ready-to-use context is then returned to the test.

Why it matters:

The caller receives a fully initialized `ApplicationDbContext` instead of having to perform database setup itself.

Byte 6: Creating a Valid Application for Tests

Builds on: Byte 1

In plain terms:

`ValidApplication()` provides a ready-made `Models.Application` object containing values that satisfy the application's normal requirements.

The code:

```
public static Models.Application ValidApplication() => new()  
{  
    Company = "Acme Labs",  
    Role = "Backend Intern",  
    Status = "Applied",  
    AppliedOn = new DateTime(2026, 9, 16),  
    JobUrl = "https://example.com/job/123",  
    Notes = "Test notes",  
    CreatedAt = DateTime.UtcNow  
};
```

What's happening:

The method uses C#'s object initializer syntax to create an application with realistic test values.

A test can simply write:

```
db.Applications.Add(TestHelpers.ValidApplication());
```

Why it matters:

Tests don't need to repeatedly construct the same valid application manually.

Byte 7: Creating a Valid Create DTO

Builds on: Byte 6

In plain terms:

ValidCreateDto() creates valid data specifically for the API's **create request**. A DTO (Data Transfer Object) is an object used to carry data between parts of the application.

The code:

```
public static DTOs.CreateApplicationDto ValidCreateDto() => new()
{
    Company = "Acme Labs",
    Role = "Backend Intern",
    Status = "Applied",
    AppliedOn = new DateTime(2026, 9, 16),
    JobUrl = "https://example.com/job/123",
    Notes = "Test notes"
};
```

What's happening:

Unlike ValidApplication(), this creates a CreateApplicationDto, not the database model.

This is the kind of object the controller's Create() method expects.

Why it matters:

It lets controller tests easily simulate a valid POST request payload without manually constructing the DTO every time.

Byte 8: Creating a Valid Update DTO

Builds on: Byte 7

In plain terms:

ValidUpdateDto() provides valid data for testing the API's update operation.

The code:

```
public static DTOs.UpdateApplicationDto ValidUpdateDto() => new()  
{  
    Company = "Contoso",  
    Role = "Offer Role",  
    Status = "Offer",  
    AppliedOn = new DateTime(2026, 9, 5),  
    JobUrl = "https://example.com/offer",  
    Notes = "Updated notes"  
};
```

What's happening:

The values are intentionally different from the original ValidApplication() values.

For example:

Acme Labs → Contoso

Applied → Offer

This makes it easy for a test to verify that an update actually changed the record.

Why it matters:

Tests can clearly distinguish the original data from the updated data, making PUT/update behavior easier to verify.

ValidationTests.cs

Byte 1: Purpose of ValidationTests

Builds on: None — starting point

In plain terms:

ValidationTests checks whether the application's validation rules correctly accept valid data and reject invalid data. These tests focus on rules such as company length, role length, status, dates, URLs, and notes.

The code:

```
public class ValidationTests
{
    // Validation rules reused from the original implementation
}
```

What's happening:

Each [Fact] method represents one independent validation scenario.

Why it matters:

Validation prevents incorrect data from entering the application. These tests make sure those rules continue working if the code changes later.

Byte 2: The Validate Helper

Builds on: Byte 1

In plain terms:

Validate() is a reusable helper that runs the model's **DataAnnotations validation** and returns any validation errors.

The code:

```
private static IList<ValidationResult> Validate(object app)
{
    var context = new ValidationContext(app, mockServices.Object, null);
    var results = new List<ValidationResult>();

    Validator.TryValidateObject(
        app,
        context,
        results,
```

```
        validateAllProperties: true);

    return results;
}
```

What's happening:

Validator.TryValidateObject() examines attributes such as [Required], [StringLength], and [Url] attached to the model or DTO.

results receives any validation errors.

Why it matters:

Instead of repeating validation setup in every test, all tests can simply call:

```
Validate(app)
```

Byte 3: Why Moq Is Used

Builds on: Byte 2

In plain terms:

ValidationContext can accept an IServiceProvider. This test doesn't need real services, so **Moq** creates a lightweight fake one.

The code:

```
var mockServices = new Mock<IServiceProvider>();
```

```
mockServices
    .Setup(s => s.GetService(It.IsAny<Type>()))
    .Returns((object?)null);
```

What's happening:

Mock<IServiceProvider> creates a fake implementation. Whenever validation asks for a service, the mock simply returns null.

Why it matters:

The tests can construct ValidationContext without starting the application's full dependency-injection system.

Byte 4: Valid Application Should Pass

Builds on: Byte 2

In plain terms:

A correctly populated application should produce **zero validation errors**.

The code:

```
[Fact]
public void Valid_application_passes_validation()
{
    Assert.Empty(
        Validate(TestHelpers.ValidApplication()));
}
```

What's happening:

ValidApplication() from TestHelpers creates known-good data. Validate() checks it, and Assert.Empty() confirms there are no errors.

Why it matters:

This establishes the basic expectation that valid data should be accepted.

Byte 5: Valid Create DTO Should Pass

Builds on: Byte 4

In plain terms:

Validation isn't only applied to the database model. The API's create DTO must also accept valid input.

The code:

```
[Fact]
public void Valid_create_dto_passes_validation()
{
    Assert.Empty(
        Validate(TestHelpers.ValidCreateDto()));
}
```

What's happening:

A valid CreateApplicationDto is passed through the same validation mechanism.

Why it matters:

It confirms the input object used by the POST endpoint has valid validation rules.

Byte 6: Wishlist Can Have No Applied Date

Builds on: Byte 4

In plain terms:

There is a special business rule: an application with status Wishlist does not need an application date.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Wishlist";
```

```
app.AppliedOn = null;
```

```
Assert.Empty(Validate(app));
```

What's happening:

The test deliberately removes AppliedOn, but changes the status to Wishlist. The validation should still succeed.

Why it matters:

This tests a **conditional validation rule**, not just a simple required field.

Byte 7: Blank Company Is Rejected

Builds on: Byte 4

In plain terms:

The company name cannot be empty.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Company = "";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The test starts with valid data, replaces the company with an empty string, and expects at least one validation error.

Why it matters:

It prevents applications from being stored without a company name.

Byte 8: Blank Role Is Rejected

Builds on: Byte 7

In plain terms:

The job role is also required, including protection against whitespace-only values.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Role = "  ";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The role contains only spaces. The validation system should treat this as invalid.

Why it matters:

Without this rule, someone could technically submit a role that looks filled in but contains no meaningful text.

Byte 9: Minimum Company and Role Length

Builds on: Byte 7 and Byte 8

In plain terms:

Company and role names must contain at least **2 characters**.

The code:

```
app.Company = "A";
```

```
Assert.NotEmpty(Validate(app));
```

```
app.Role = "x";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

Each test supplies a one-character value and expects validation to fail.

Why it matters:

This prevents unrealistically short company or role values from being accepted.

Byte 10: Maximum Company and Role Length

Builds on: Byte 9

In plain terms:

Company and role names also have an upper limit of **100 characters**.

The code:

```
app.Company = new string('a', 101);
```

```
Assert.NotEmpty(Validate(app));
```

```
app.Role = new string('b', 101);
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

`new string('a', 101)` creates a string containing 101 a characters. Since the limit is 100, validation should fail.

Why it matters:

It prevents excessively large values from being accepted for fields intended to contain short names.

Byte 11: Notes Have a 1000-Character Limit

Builds on: Byte 10

In plain terms:

The optional Notes field can contain text, but it cannot exceed **1000 characters**.

The code:

```
app.Notes = new string('n', 1001);
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The test creates exactly 1001 characters, crossing the documented 1000-character limit.

Why it matters:

It verifies the maximum length constraint on a larger free-text field.

Byte 12: Status Must Be Allowed

Builds on: Byte 4

In plain terms:

The application status must come from the application's allowed set. An unknown value such as "Hired" should fail validation.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Hired";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The test replaces the valid "Applied" status with "Hired" and expects a validation error.

Why it matters:

This keeps status values consistent throughout the application.

Byte 13: Applied Date Is Conditionally Required

Builds on: Byte 6

In plain terms:

The opposite of the Wishlist rule is also tested: if the status is not Wishlist, AppliedOn must be present.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Applied";
```

```
app.AppliedOn = null;
```

```
var errors = Validate(app);
```

```
Assert.Contains(  
    errors,  
    e => e.MemberNames.Contains("AppliedOn"));
```

What's happening:

The test removes the date while keeping the status as Applied. It specifically checks that the validation error belongs to AppliedOn.

Why it matters:

This confirms the conditional rule is working in both directions.

Byte 14: Invalid Job URL Is Rejected

Builds on: Byte 4

In plain terms:

If a job URL is supplied, it must be a valid URL format.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.JobUrl = "not-a-url";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The test replaces the valid URL with plain text that isn't a URL and expects validation to fail.

Why it matters:

It prevents malformed job links from being accepted.

Byte 15: Optional Fields Can Be Empty

Builds on: Byte 14

In plain terms:

JobUrl and Notes are optional fields. Leaving them empty should not make an otherwise valid application invalid.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.JobUrl = null;
```

```
app.Notes = null;
```

```
Assert.Empty(Validate(app));
```

What's happening:

Both optional fields are set to null, while the required fields remain valid. The validation result must contain no errors.

Why it matters:

It distinguishes **optional fields** from required fields and ensures users aren't forced to provide unnecessary information.

AFTER IMPLEMENTING NGINX

Byte 1: NGINX's Role in the Project

Builds on: None — starting point

In plain terms:

NGINX is placed in front of the existing Vue frontend and ASP.NET Core API. It acts as a **reverse proxy**, meaning it receives the browser's request and forwards it to the correct internal application.

The code:

```
server {  
    listen 8080;  
}
```

What's happening:

NGINX becomes the single entry point for the application and listens on port 8080.

Why it matters:

Instead of the user accessing Vue and the API separately, the user can access the application through one URL:

http://localhost:8080

Byte 2: The Project Uses a Local NGINX Setup

Builds on: Byte 1

In plain terms:

NGINX has been added directly inside the project rather than relying on a system-wide installation.

The code:

```
nginx/  
├── conf/  
│   └── nginx.conf  
├── nginx.exe  
├── start-nginx.ps1  
├── stop-nginx.ps1  
└── logs/
```

What's happening:

The project contains the NGINX executable, configuration file, startup script, shutdown script, and logs.

Why it matters:

The project can control its own NGINX setup without changing the existing Vue or ASP.NET Core architecture.

Byte 3: NGINX Listens on Port 8080

Builds on: Byte 1

In plain terms:

Port 8080 is the **public entry point** of this local application.

The code:

```
listen 8080;
```

What's happening:

When you open:

```
http://localhost:8080
```

the request first reaches NGINX.

Why it matters:

The user only needs to know the NGINX port rather than separately accessing the frontend and backend ports.

Byte 4: Normal Requests Go to Vue

Builds on: Byte 3

In plain terms:

NGINX checks the URL path. Requests to / are forwarded to the Vue development server running on port 5173.

The code:

```
location / {  
    proxy_pass http://127.0.0.1:5173;  
}
```

What's happening:

Browser

↓

localhost:8080/

↓

NGINX

↓

127.0.0.1:5173

↓

Vue

Why it matters:

NGINX doesn't serve the Vue application itself. It forwards the request to the existing Vue server.

Byte 5: API Requests Go to ASP.NET Core

Builds on: Byte 4

In plain terms:

Requests beginning with /api/ are treated differently. NGINX forwards them to the ASP.NET Core API on port 5038.

The code:

```
location /api/ {  
    proxy_pass http://127.0.0.1:5038;  
}
```

What's happening:

Browser

↓

localhost:8080/api/applications

↓

NGINX

↓

127.0.0.1:5038

↓

ASP.NET Core API

The API then continues its normal processing with EF Core and PostgreSQL.

Why it matters:

This allows both the frontend and backend to be accessed through the same NGINX port.

Byte 6: NGINX Does Not Replace the Existing Application

Builds on: Bytes 4–5

In plain terms:

NGINX is only an additional routing layer. Your existing Vue, ASP.NET Core API, EF Core, and PostgreSQL implementation remains responsible for its original jobs.

The code:

NGINX :8080

└── → Vue :5173

└── → API :5038

↓

PostgreSQL :5432

What's happening:

NGINX does not perform CRUD, database operations, or business logic.

Why it matters:

If NGINX is removed, the underlying Vue and API can still exist independently. NGINX simply provides a single entry point.

Byte 7: The Startup Script Controls NGINX

Builds on: Byte 3

In plain terms:

The project includes a PowerShell script for starting/reloading NGINX.

The code:

nginx/start-nginx.ps1

The report also shows that this script prints:

NGINX started/reloaded: http://localhost:8080

What's happening:

Instead of manually typing all NGINX commands every time, the project provides a script to start or reload the proxy.

Why it matters:

It makes starting the project's NGINX setup easier and repeatable.

Byte 8: NGINX Can Be Stopped Separately

Builds on: Byte 7

In plain terms:

There is also a dedicated script for stopping NGINX.

The code:

```
nginx/stop-nginx.ps1
```

What's happening:

This allows NGINX to be stopped without changing or stopping the actual Vue/API source code.

Why it matters:

It separates NGINX process management from the application itself.

Byte 9: Logs Prove the Proxy Has Worked

Builds on: Bytes 4–5

In plain terms:

The NGINX access log contains evidence that requests were previously successfully proxied to both Vue and the API.

The evidence:

```
GET / → 200
```

```
GET /api/applications → 200
```

What's happening:

A 200 response means those requests successfully reached the appropriate upstream service and returned successfully during that earlier run.

The report also mentions earlier 502 responses when the upstream services were not running.

Why it matters:

A 502 Bad Gateway here indicates NGINX was running but couldn't reach the upstream service. Once the Vue/API services were available, successful 200 responses were recorded.

Byte 10: Current NGINX State

Builds on: Bytes 1–9

In plain terms:

The NGINX implementation **exists and has previously worked**, but it is currently stopped.

The evidence:

```
Get-Process -Name nginx = none
```

and:

```
No listener on port 8080
```

and:

```
http://localhost:8080/
```

→ Unable to connect

What's happening:

The configuration is implemented, but the NGINX process is not currently running.

Why it matters:

This is a **runtime state**, not an implementation failure. Starting NGINX should make localhost:8080 available, provided Vue and the API are also running.

Byte 11: The Stale PID File

Builds on: Byte 10

In plain terms:

The project has an old NGINX process-ID file:

nginx/logs/nginx.pid

→ 22232

but there is currently no NGINX process running.

What's happening:

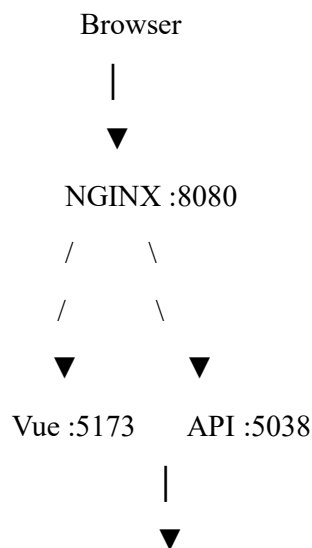
A PID file normally records the process ID of NGINX. Here, the file appears to be left over from an earlier run.

Why it matters:

A stale PID file can sometimes cause confusion when starting/stopping NGINX, so the startup script or NGINX configuration needs to handle it correctly.

PUTTING IT TOGETHER

Your existing application is now structured as:



EF Core + Npgsql



PostgreSQL :5432

NGINX provides the **single entry point on port 8080**. / is forwarded to Vue, while /api/ is forwarded to ASP.NET Core. The report confirms that this routing **has already successfully produced 200 responses**, but NGINX is currently stopped.

NGINX acts as a traffic controller/reverse proxy.

Browser



NGINX :8080

└─ / → Vue :5173

└─ /api/ → ASP.NET API :5038



PostgreSQL

How it works

1. User opens **localhost:8080**.
2. NGINX receives the request.
3. If the request is /, NGINX sends it to **Vue (5173)**.
4. If the request is /api/..., NGINX sends it to **ASP.NET Core (5038)**.
5. The response comes back through NGINX to the browser.

Viva answer:

“NGINX provides one entry point on port 8080 and routes frontend requests to Vue and API requests to ASP.NET Core.”